1 Introduction

This document discusses camera calibration using OpenCV. It covers the **perspective** and **fisheye** models.

See the attached Python code and test images in `calibration_tmorley.zip`.

2 Perspective Model

2.1 Canon 18mm

Run test script to calibrate:

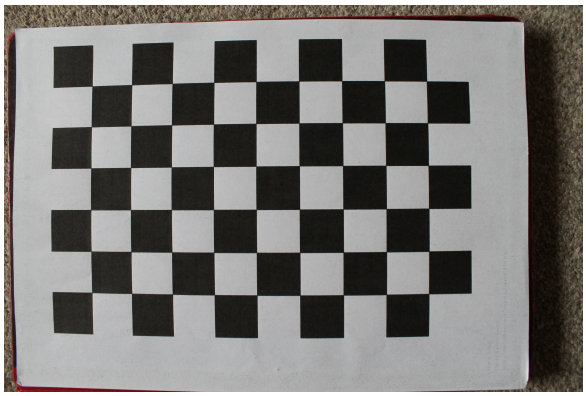
```
cd calibration_tmorley/basic_model/
python3 calibrate.py 'canon_18mm_cal/*.JPG' canon18mm
```

This finds the camera parameters using the chessboard images in `canon_18mm_cal/*.JPG`. The supplied model name (canon18mm) is used in the file names that are saved in the current directory: `canon18mm_matrix.txt` and `canon18mm_distortion.txt`.

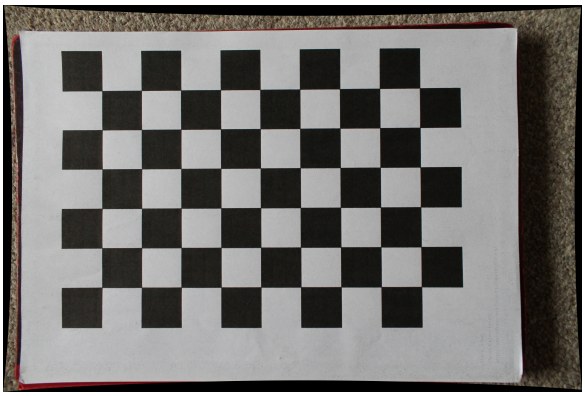
Run test script to undistort:

```
python3 undistort.py canon_18mm_test/test.jpg canon18mm
```

This takes the camera parameter files that were created in the calibration stage and uses them to undistort the supplied image (`canon_18mm_test/test.jpg`). The output image is saved in the current directory as `calibresult.jpg`.



Distorted



Undistorted

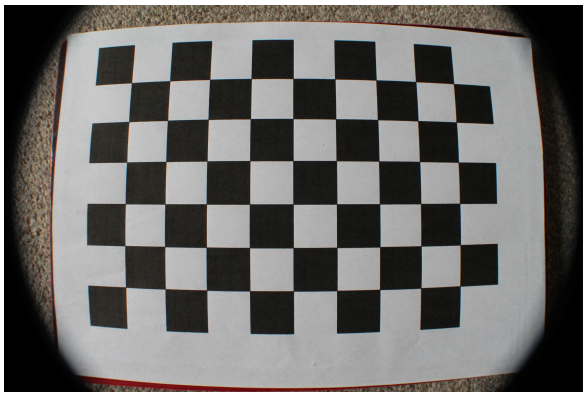
2.2 Canon 8mm

Run test script to calibrate:

```
cd calibration_tmorley/basic_model/
python3 calibrate.py 'canon_8mm_cal/*.JPG' canon8mm
```

Run test script to undistort:

```
python3 undistort.py canon_8mm_test/test.jpg canon8mm
```

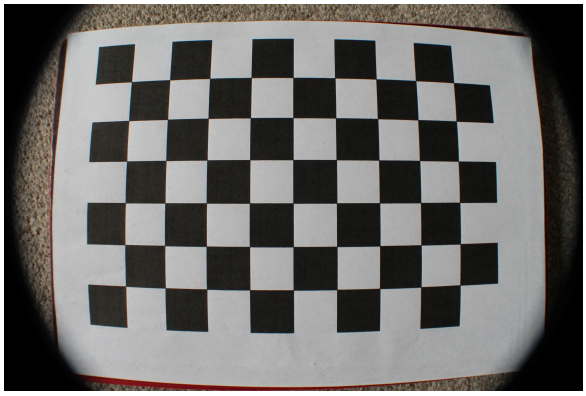


Distorted

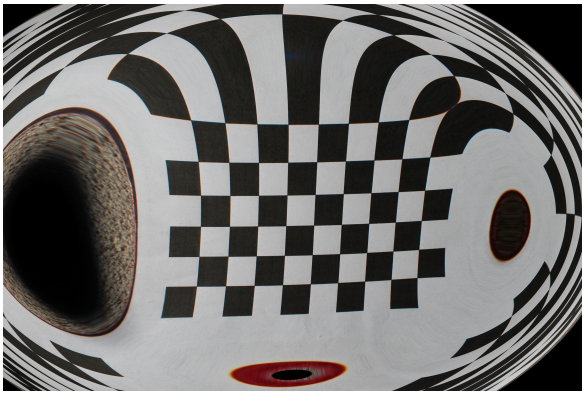
Undistorted

This obviously failed.

Before this, with the same code, I calibrated with the images contained in `calibration/basic_model/canon_8mm_test_POOR_CAL` and then ran the undistort on the test image and the result is shown below.



Distorted



Undistorted

The undistort script returned a Region of interest = (0, 0, 0, 0). This is normally due to the calibration images not having enough chessboard corners close to the edge of the images. It needs this because this is where the most distortion is. (This also happened on an 18mm calibration because the chessboards were smallish in the images.) So new chessboard images were taken and that resulted in the output given in that section.

3 Fisheye Model

3.1 Canon 8mm

Run test script to calibrate:

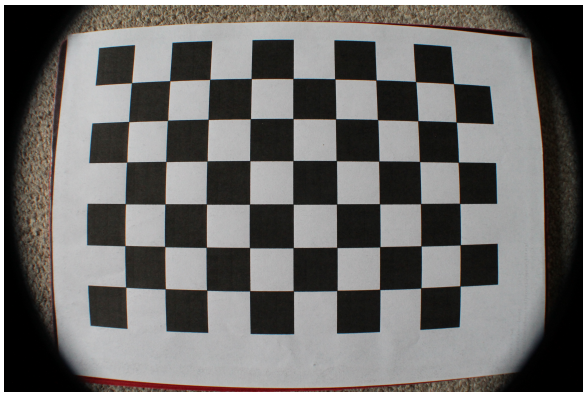
```
cd calibration_tmorley/fisheye_model/
python3 calibrate.py 'canon_8mm_cal/*.JPG' canon8mm
```

This finds the camera parameters using the chessboard images in `canon_8mm_cal/*.JPG`. The supplied model name (canon8mm) is used in the file names that are saved in the current directory: `canon8mm_matrix.txt` and `canon8mm_distortion.txt`.

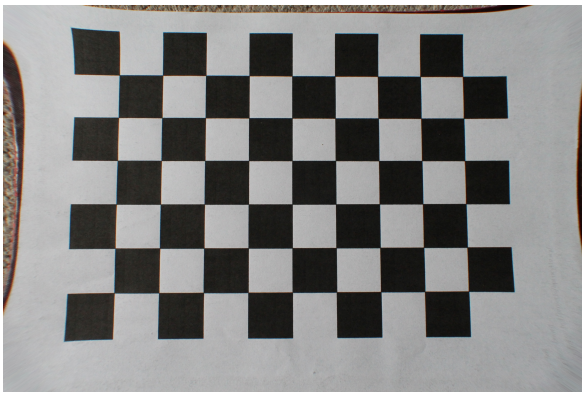
Run test script to undistort:

```
python3 undistort.py canon_8mm_test/test.jpg canon8mm
```

This takes the camera parameter files that were created in the calibration stage and uses them to undistort the supplied image (`canon_8mm_test/test.jpg`). The output image is saved in the current directory as `calibresult.jpg`.



Distorted



Undistorted

This is not too bad, but at the very top-left and bottom-left are still distorted. This is exactly what is shown on the OpenCV website for the problem with the perspective model and how the fisheye model solves it. But this is not the case here: the perspective model doesn't work for this lens and it is the fisheye model that has this error.

See undistortImage() in `https://docs.opencv.org/master/db/d58/group__calib3d__fisheye.html`. But while we are on the subject of undistortImage(), the help says 'The function is simply a combination of fisheye::initUndistortRectifyMap and remap.' This function doesn't work for me and I had to use initUndistortRectifyMap and remap separately.

3.2 Canon 8mm - Attempt 2

This time, a second set of chessboard images were taken, but with the chessboard large in the frame and making movements of the camera to keep the squares close to the edges of the frame.

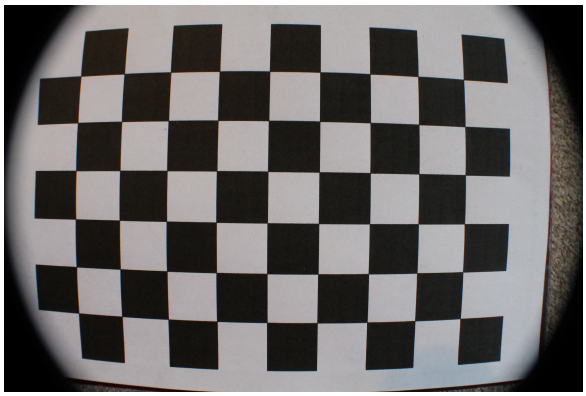
Run test script to calibrate:

```
cd calibration_tmorley/fisheye_model/
python3 calibrate.py 'canon_8mm_cal_2/*.JPG' canon8mm_2
```

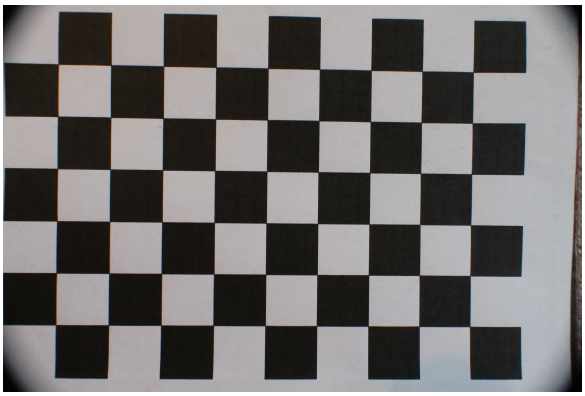
NOTE Out of the 12 chessboard images, the chessboard corners were only found in 7 of them. This is despite the full chessboard being visible in each frame. (Also note that the function takes a long time looking for the corners in the ones it fails on, making the script take a longer time to run.)

Run test script to undistort:

```
python3 undistort.py canon_8mm_test_2/test.jpg canon8mm_2
```



Distorted



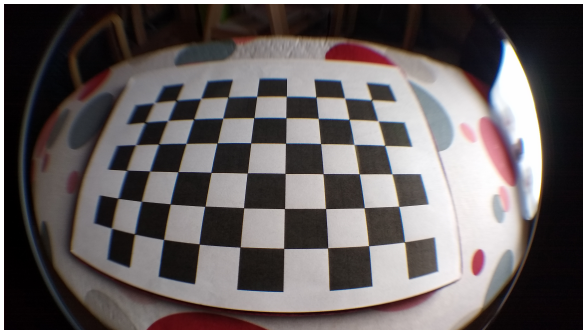
Undistorted

So this time, **a good job was done**. The code was the same – just a different choice of chessboard images were taken.

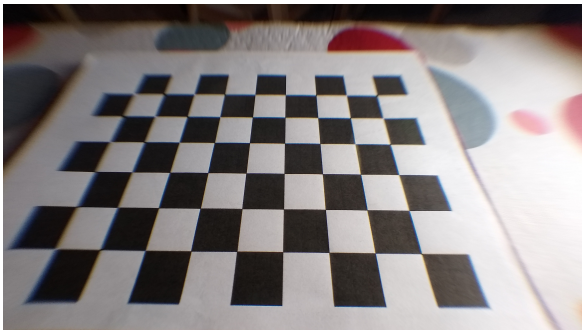
3.3 Mobile Phone Fisheye

This was using a clip-on fisheye lens on a mobile phone camera and the same code as previously. The model name in the example data is phone_fisheye. The edges are quite blurred using this lens.

Out of the 9 chessboard images only 2 were good enough for the script to find corners in.



Distorted

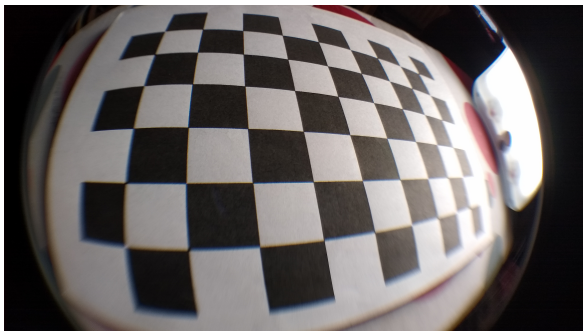


Undistorted

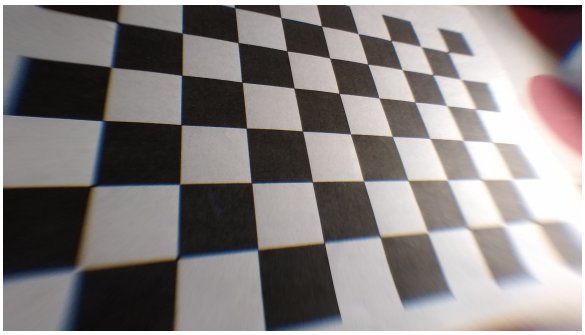
3.4 Mobile Phone Fisheye Attempt 2

As above but with a different set of chessboard images.

Out of the 11 chessboard images only 5 were good enough for the script to find corners in.



Distorted



Undistorted

3.5 Mobile Phone Fisheye Photos

The example data has a directory with some photos taken with the fisheye lens on the mobile phone. Below is one example and notice that the second output image is a bit straighter than the other - the better one is the one using the calibration from Section 3.4.

Distorted

Undistorted with Calibration 1

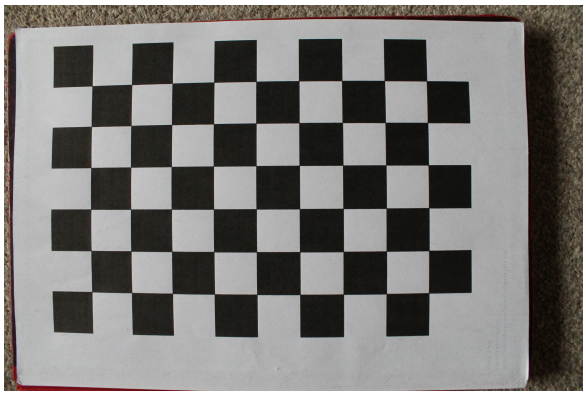
Undistorted with Calibration 2

The images are very blurred but this is probably a result of the input image. The images produced by this £5 clip-on phone lens are of very poor quality (I'll pay more money out in future).

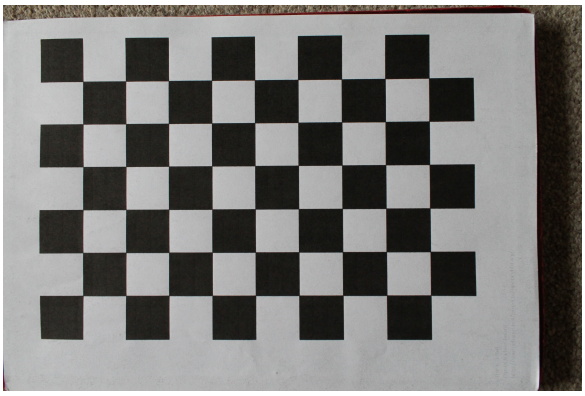
Also notice the tight cropping and that the horizon isn't present in the output images (see Section 4).

3.6 Canon 18mm Using Fisheye Model

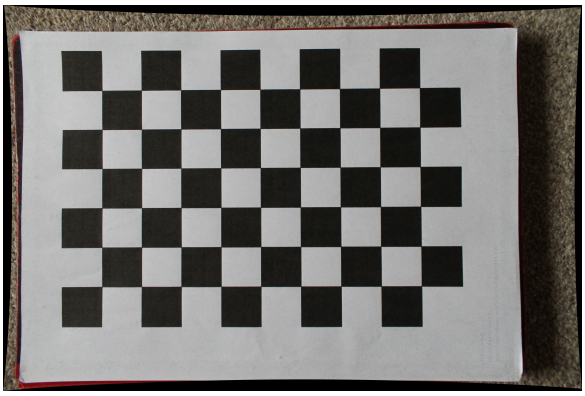
This trial is to see how the non-fisheye images from the 18mm Canon respond to the fisheye model.



Distorted



Undistorted



Undistorted by perspective model

This shows that the fisheye model works with non-fisheye lens.

But notice the tight cropping that is done by the fisheye script when compared to the one from the perspective model. This can also be seen on the fisheye examples above. See Section 4.

4 Fisheye Cropping Problem

It was seen in the undistorted fisheye images that the cropping was very tight. In the perspective model the region of interest is returned and this can be used to crop the image:

```
newcameramtx, roi = cv.getOptimalNewCameraMatrix(mtx, dist, (w,h), 1, (w,h))
print('Region of interest =', roi)
dst = cv.undistort(img, mtx, dist, None, newcameramtx)

# crop the image
x, y, w, h = roi
#dst = dst[y:y+h, x:x+w]
```

I had commented out the cropping so that I could see attempted undistorted images even if the calibration was poor. But if I use the specified cropping on a good calibration, the specified cropping is ok anyway.

But in the fisheye model, a region of interest isn't returned:

```
map1, map2 = cv2.fisheye.initUndistortRectifyMap(K, D, np.eye(3), K, DIM, cv2.CV_16SC2)
undistorted_img = cv2.remap(img, map1, map2, interpolation=cv2.INTER_LINEAR,
                borderMode=cv2.BORDER_CONSTANT)
```

Note that the output image size is the same size as the input, it is just that the region of interest is scaled to fit.

The focal length and output image size for the remapping can be altered to produce more of the image but it just seems to bring in the areas that the warping couldn't accommodate. So the functions are just outputting the best part of the undistorted image.

See the following website for more information:
`https://medium.com/@kennethjiang/calibrate-fisheye-lens-using-opencv-part-2-13990f1b157f`

5 Conclusion

The perspective model on non-fisheye lens works very well. On wide-angle lenses, it fails.

For the fisheye model, a good result for a wide-angle lens in shown in Section 3.2. This was achieved by a good choice of chessboard image which shows that it might be worth trying different sets of calibration images to see which perform the best. The OpenCV functions rejected some of the chessboard images but they appeared to look ok with all the chessboard corners visible.

On an actual fisheye lens (the phone camera ones) the fisheye model performed well. The output images were very blurred and locally distorted, but this might have just been because of the very cheap clip-on fisheye lens – the geometry looked to be properly corrected. See Section 3.5.

The fisheye functions seem to perform quite tight cropping, but I think that this is just to output the best part of the undistorted image. I tried to get it to output more of the image but it just introduced black areas that the mapping/image were unable to fill.